

EE671: FOUNDATION OF VLSI CAD

ASSIGNMENT – 1

Logic Simulator for a Combinational Circuit

Name: Dharavath Nikhitha

Roll Number: 26M1167

1. Problem Statement

Design a logic simulator which can simulate a combinational circuit.

The input to the simulator is a gate-level netlist. For this assignment, the Verilog (.v) format is used. The output is a file containing the logical values produced by the circuit.

The main purpose of this assignment is to read the gate connections, find the correct order of evaluation, and simulate the circuit for the given input values.

2. Implementation Used

A Python program is used as the logic simulator. The test circuit is a 1-bit full adder made using nine 2-input NAND gates. The simulator reads the Verilog file, identifies the gates and their connections, levelizes the gates, and then evaluates them for each input vector.

The program uses only Python standard libraries. No external Python package is required.

3. Files Included

File	Purpose
logic_simulator.py	Main Python program. It parses the Verilog file, builds gate dependencies, levelizes the circuit and performs simulation.
fulladder.v	Gate-level Verilog description of the 1-bit full adder using 9 NAND gates.
input_vectors.txt	Contains the input combinations to be tested.
output_results.txt	Contains the input vector and the corresponding simulated output.

All four files are kept in the same folder so that the program can be run directly from the terminal.

4. Circuit Used for Testing

The test circuit is a 1-bit full adder. It has three inputs: a, b and cin. It has two outputs: sum and carry. The circuit is implemented only using NAND gates.

The Verilog file contains the following nine gate instances:

```
nand g1 (w1, a, b);
nand g2 (w2, a, w1);
nand g3 (w3, b, w1);
nand g4 (w4, w2, w3);
nand g5 (w5, w4, cin);
nand g6 (w6, w4, w5);
nand g7 (w7, cin, w5);
nand g8 (sum, w6, w7);
nand g9 (carry, w5, w1);
```

The internal wires w1 to w7 connect the outputs of one gate to the inputs of other gates.

5. Algorithm of the Program

The complete algorithm is simple: read the circuit → store the gates → find dependencies → levelize the gates → give input values → evaluate the gates → obtain the outputs → save the results.

5.1 Read the Verilog Netlist

The program opens the .v file and reads its contents. Verilog comments are removed first so that comments do not affect parsing.

5.2 Find Inputs, Outputs and Gates

The program finds the primary inputs and primary outputs from the module declarations. It then finds each supported gate and records its gate type, gate name, output signal and input signals.

5.3 Store Gate Information

Each gate is stored as an object of the Gate class. This makes it easy to keep all information about one gate together.

5.4 Build Gate Dependencies

The program checks which gate produces each signal. If a gate needs a signal produced by another gate, the first gate must be evaluated before the second gate. This relation is stored as a dependency.

5.5 Levelize the Gates

Levelization gives the correct order for evaluating the gates. Gates whose inputs are already available are placed in an earlier level.

For the present circuit, the levelization is:

Level	Gates
1	g1
2	g2, g3
3	g4
4	g5
5	g6, g7, g9
6	g8

For example, g2 and g3 use w1, which is produced by g1. Therefore, g1 must be evaluated first.

5.6 Simulate an Input Vector

The input bits are assigned to a, b and cin. The gates are then evaluated level by level. When a gate is evaluated, its input signal values are taken from the stored signal values. The calculated value is stored as the output of that gate.

5.7 Generate the Output

After all gates are evaluated, the values of sum and carry are read in that order. These two bits form the final output vector.

5.8 Process Multiple Vectors

In batch mode, the program reads all the vectors from `input_vectors.txt`, simulates each one, and writes the corresponding results to `output_results.txt`.

6. Data Structures Used

The program uses simple Python data structures. Each one has a specific purpose.

Data Structure	Use in the Program
Gate class	Represents one logic gate and stores its type, name, output, inputs, level and value.
List	Stores gates, primary inputs, primary outputs, input vectors and simulation results.
Dictionary	Stores signal values, the gate producing each signal, and dependency information.
Set	Stores unique dependencies and dependent gates.
Queue / list	Used during levelization to process gates whose dependencies are completed.
defaultdict(list)	Groups gates according to their level.

7. Gate Class

The Gate class is the main data structure of the simulator. For every gate, the following information is stored:

- `gate_type` – type of gate, such as NAND.
- `name` – gate name, such as g1.
- `output_signal` – signal produced by the gate, such as w1.
- `input_signals` – signals connected to the gate inputs.
- `level` – level assigned during levelization.
- `value` – output value obtained after evaluation.

The `evaluate()` method of the Gate class performs the Boolean operation. The program supports AND, OR, NAND, NOR, XOR, XNOR, NOT and BUF gates.

8. Important Functions in the Python Program

Function	Purpose
remove_comments()	Removes // and /* ... */ comments from the Verilog file.
clean_signal()	Cleans signal names before they are stored.
parse_declarations()	Finds input and output signal declarations.
parse_verilog_netlist()	Reads the Verilog netlist and creates Gate objects.
build_graph()	Creates the driver map and gate dependency relationships.
levelize_gates()	Finds the correct gate evaluation levels.
simulate_vector()	Simulates one input vector.
read_vectors()	Reads input vectors from the input file.
simulate_from_vector_file()	Simulates all vectors and writes the results.
main()	Controls the complete execution of the program.

9. How to Run the Program

No special configuration is required. Python must be installed on the computer.

1. Keep these files in the same folder: logic_simulator.py, fulladder.v, input_vectors.txt and output_results.txt (the output file may be created automatically).
2. Open Command Prompt or PowerShell in that folder.
3. Check Python installation using:

```
python --version
```

4. Run the complete batch simulation using:

```
python logic_simulator.py fulladder.v input_vectors.txt output_results.txt
```

5. The program displays the primary inputs, primary outputs, number of gates, gate information and gate levelization.
6. It then simulates all vectors in input_vectors.txt.
7. The final results are saved in output_results.txt.

9.1 Single-Vector Mode

A single input vector can also be tested without an input file:

```
python logic_simulator.py fulladder.v 101
```

For input 101, the program produces output 01.

10. Input File

The input_vectors.txt file used in this assignment contains all eight possible combinations of the three input bits:

```
000
001
010
011
100
101
110
111
```

Each line represents one complete input vector in the order a, b, cin.

11. Output File

The output_results.txt file generated by the program is:

```
000 --> 00
001 --> 10
010 --> 10
011 --> 01
100 --> 10
101 --> 01
110 --> 01
111 --> 11
```

The output bits are in the order sum, carry.

13. Example of One Simulation

Consider the input vector 101. Therefore, $a = 1$, $b = 0$ and $cin = 1$.

- g1 produces $w1 = 1$.
- g2 and g3 are evaluated next.
- g4 produces $w4 = 1$.
- g5 produces $w5 = 0$.
- g6, g7 and g9 are then evaluated. g9 produces $carry = 1$.
- Finally, g8 produces $sum = 0$.

Therefore, for input 101, the output is 01.

14. Program Execution Screenshots

```
PS C:\Users\nikhi> cd "C:\Users\nikhi\Documents"
PS C:\Users\nikhi\Documents> python logic_simulator.py fulladder.v input_vectors.txt output_results.txt

=====
LOGIC SIMULATOR
=====
Primary Inputs : a, b, cin
Primary Outputs: sum, carry
Number of Gates: 9

=====
GATE INFORMATION
=====
g1  : NAND Output = w1      Inputs = a, b
g2  : NAND Output = w2      Inputs = a, w1
g3  : NAND Output = w3      Inputs = b, w1
g4  : NAND Output = w4      Inputs = w2, w3
g5  : NAND Output = w5      Inputs = w4, cin
g6  : NAND Output = w6      Inputs = w4, w5
g7  : NAND Output = w7      Inputs = cin, w5
g9  : NAND Output = carry   Inputs = w5, w1
g8  : NAND Output = sum     Inputs = w6, w7

=====
GATE LEVELIZATION
=====
Level 1: g1
Level 2: g2, g3
Level 3: g4
Level 4: g5
Level 5: g6, g7, g9
Level 6: g8
=====
```

Batch-mode execution showing the detected inputs, outputs, number of gates and levelization of gates.

```
=====
SIMULATION RESULTS
=====
000 --> 00
001 --> 10
010 --> 10
011 --> 01
100 --> 10
101 --> 01
110 --> 01
111 --> 11
=====

Results successfully saved to:
output_results.txt
PS C:\Users\nikhi\Documents> 
```

Simulation result for batch-mode execution

```

=====
GATE INFORMATION
=====
g1  : NAND Output = w1      Inputs = a, b
g2  : NAND Output = w2      Inputs = a, w1
g3  : NAND Output = w3      Inputs = b, w1
g4  : NAND Output = w4      Inputs = w2, w3
g5  : NAND Output = w5      Inputs = w4, cin
g6  : NAND Output = w6      Inputs = w4, w5
g7  : NAND Output = w7      Inputs = cin, w5
g9  : NAND Output = carry   Inputs = w5, w1
g8  : NAND Output = sum     Inputs = w6, w7

=====

GATE LEVELIZATION
=====
Level 1: g1
Level 2: g2, g3
Level 3: g4
Level 4: g5
Level 5: g6, g7, g9
Level 6: g8

=====

SIMULATION RESULT
=====
101 --> 01
=====

```

Single-vector simulation for input 101, producing output 01.

15. Error Checking

The program includes basic checks to avoid invalid simulation. It checks the command-line arguments, Verilog file availability, valid input-vector length, binary input values, multiple drivers for the same signal, and combinational loops that cannot be levelized.

16. Limitations

- The parser is intended for simple structural Verilog netlists.
- The simulator is mainly intended for combinational circuits.
- Sequential elements such as flip-flops and clocks are not simulated.
- The simulator uses binary 0 and 1 logic and does not model timing delays.

17. Conclusion

A Python-based logic simulator was implemented for a combinational gate-level circuit. A 1-bit full adder using nine NAND gates was used for testing. The program successfully reads the Verilog netlist, stores gate information, finds gate dependencies, levelizes the circuit, simulates input vectors and writes the output values to a file.

All eight possible input combinations were tested and the obtained outputs matched the expected full-adder results. The implementation therefore satisfies the required gate-level simulation task.

.